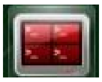
RTAB-VSLAM 3D Navigation

1. Algorithm Introduction & Principle

For an introduction to the RTAB-VSLAM algorithm and its principles, refer to "Mapping Lesson/Lesson 5 RTAB-VSLAM 3D Mapping" for study and reference.

2. Robot Operations

1）Start the robot, and access the robot system desktop using VNC.

2）Click-on  to open the command line terminal.

3）Execute the command to disable the app auto-start service:

~/.stop_ros.sh

4）Enter the following command to start the navigation service and press Enter:

ros2 launch navigation rtabmap_navigation.launch.py

3. Virtual Machine Operations

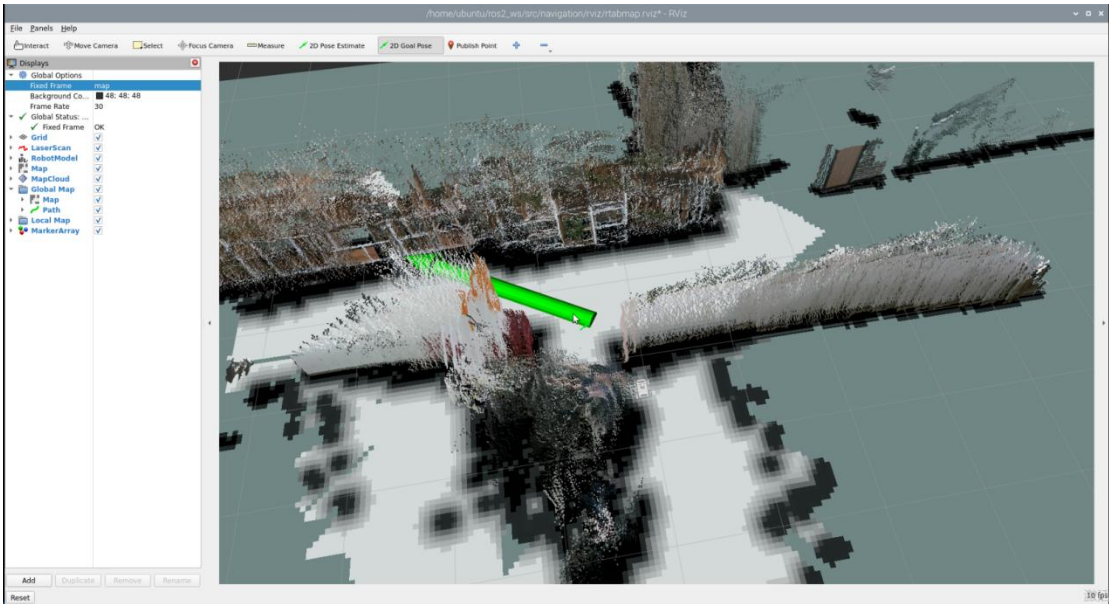
1）Click-on  to open the command-line terminal on the virtual machine.

2）Enter the following command to start the RViz tool for navigation display:

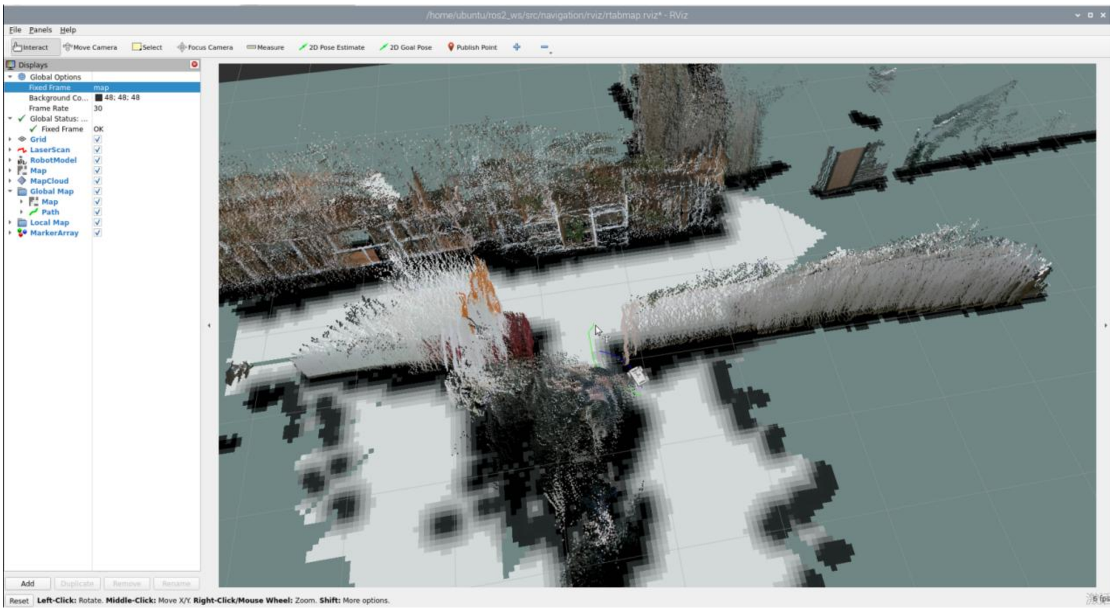
ros2 launch navigation rviz_rtabmap_navigation.launch.py

In the software menu bar, "2D Nav Goal" is used to set a single target point for the robot, and "Publish Point" is used to set multiple target points for the robot.

3) Click 2D Goal Pose , and on the map interface, select a location as the target point by clicking the mouse once at that point. After the selection is made, the robot will automatically generate a route and move to the target point.



4) After confirming the target point, the map will display two paths: a line composed of blue squares representing the straight-line path between the robot and the target point, while the dark blue line represents the planned path of the robot.



5）After encountering obstacles, the car will navigate around them and continuously adjust its posture and trajectory.

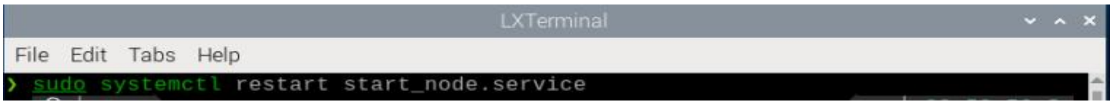
6）If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

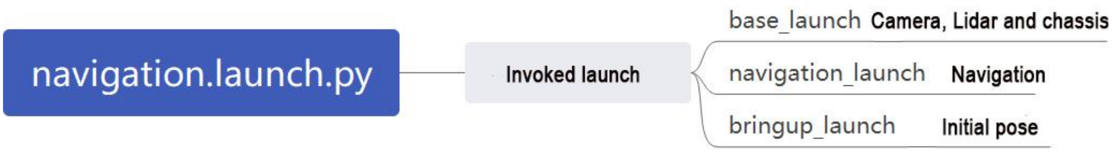
Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

sudo systemctl restart start_node.service



4. Launch Description



The launch file is saved in:

/home/ubuntu/ros2_ws/src/navigation/launch/navigation.launch.py

```
import os
from ament_index_python.packages import get_package_share_directory

from launch_ros.actions import PushRosNamespace
from launch import LaunchDescription, LaunchService
from launch.substitutions import LaunchConfiguration
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, OpaqueFunction, TimerAction

def launch_setup(context):
    compiled = os.environ['need_compile']
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
        navigation_package_path = get_package_share_directory('navigation')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
        navigation_package_path = '/home/ubuntu/ros2_ws/src/navigation'

    sim = LaunchConfiguration('sim', default='false').perform(context)
    map_name = LaunchConfiguration('map', default='map_01').perform(context)
    robot_name = LaunchConfiguration('robot_name', default=os.environ['HOST']).perform(context)
    master_name = LaunchConfiguration('master_name', default=os.environ['MASTER']).perform(context)
    use_teb = LaunchConfiguration('use_teb', default='false').perform(context)

    sim_arg = DeclareLaunchArgument('sim', default_value=sim)
    map_name_arg = DeclareLaunchArgument('map', default_value=map_name)
    master_name_arg = DeclareLaunchArgument('master_name', default_value=master_name)
    robot_name_arg = DeclareLaunchArgument('robot_name', default_value=robot_name)
    use_teb_arg = DeclareLaunchArgument('use_teb', default_value=use_teb)

    use_sim_time = 'true' if sim == 'true' else 'false'
    use_namespace = 'true' if robot_name != '/' else 'false'
```

◆ Set Paths

Obtain the paths for the “slam” and “navigation” packages.

```
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
        navigation_package_path = get_package_share_directory('navigation')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
        navigation_package_path = '/home/ubuntu/ros2_ws/src/navigation'
```

◆ Enable Other Launch Files

“rtabmap_launch” to start 3D navigation algorithm

“navigation_launch” to start 2D navigation algorithm

“bringup_launch” to initialize actions

```
    base_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(slam_package_path, 'launch/include/robot.launch.py')),
        launch_arguments={
            'sim': sim,
            'master_name': master_name,
            'robot_name': robot_name
        }.items(),
    )

    navigation_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(navigation_package_path, 'launch/include/bringup.launch.py')),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'map': os.path.join(slam_package_path, 'maps', map_name + '.yaml'),
            'params_file': os.path.join(navigation_package_path, 'config', 'nav2_params.yaml'),
            'namespace': robot_name,
            'use_namespace': use_namespace,
            'autostart': 'true',
            'use_teb': use_teb,
        }.items(),
    )

    bringup_launch = GroupAction(
     actions=[
         PushRosNamespace(robot_name),
         base_launch,
         TimerAction(
             period=10.0,  # 延时等待其它节点启动好(delay for enabling other nodes)
             actions=[navigation_launch],
         ),
      ]
    )
```